Name: K.Likitha

Hall Ticket: 2303A52144

Batch: 41

Task 1: Hospital Management Database

Prompt:

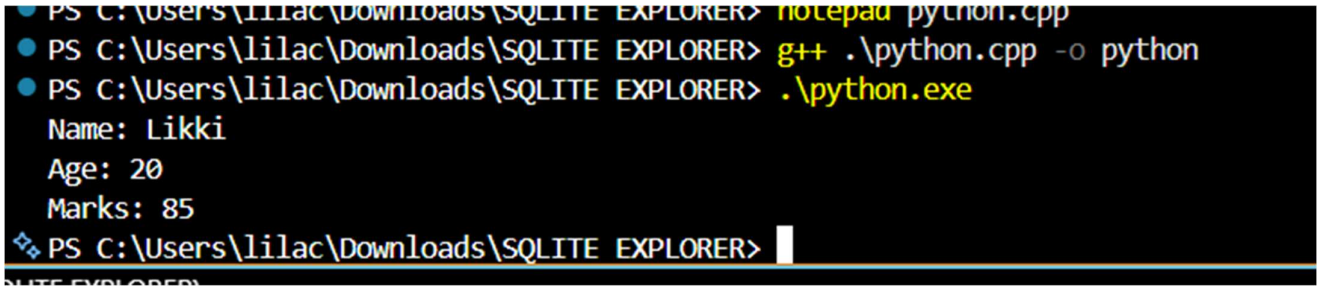
Convert the above Python Student class into equivalent C++ code. Include a main() function and produce the same output as the Python program.

Code:

```
// Python Code (for reference)
/*
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

s1 = Student("Likki", 20, 85)
s1.display()
*/

// Convert the above Python Student class into equivalent C++ code

#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int age;
    int marks;

public:
    Student(string n, int a, int m) {
        name = n;
        age = a;
        marks = m;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Marks: " << marks << endl;
    }
```

```
};

int main() {
    Student s1("Likki", 20, 85);
    s1.display();
    return 0;
}
```

Output:



Explanation:

The Python class is converted into a C++ class by defining attributes as private members and initializing them using a constructor. Member functions are used to display output, and the main() function creates an object and calls the display method.

Task 2: **Java to Python Function Conversion**

Prompt:

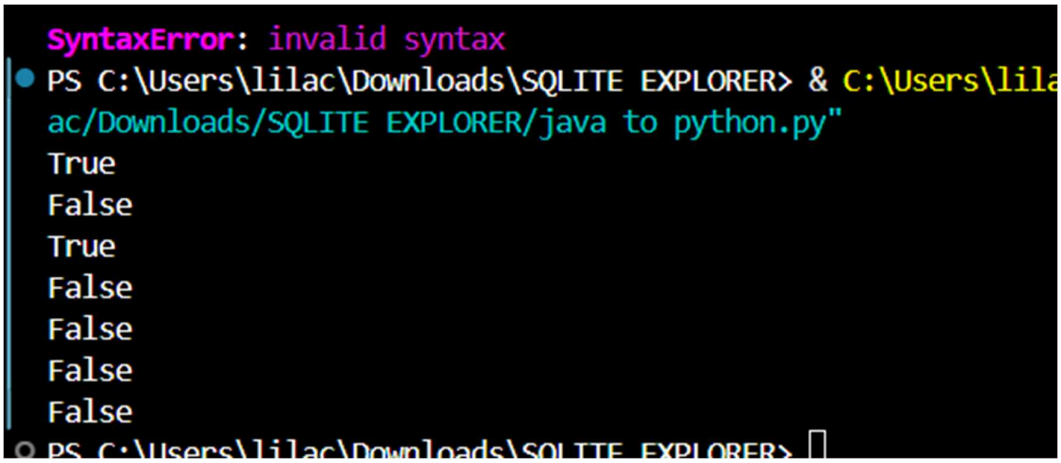
Convert the above Java isPrime() method into equivalent Python function. Ensure Pythonic syntax and test with multiple inputs.

Code:

```
# Java Code
'''
public class PrimeCheck {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= n / 2; i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isPrime(7));
        System.out.println(isPrime(10));
    }
}
*/'''
#Convert the above Java isPrime() method into equivalent Python function. Ensure Pythonic
syntax and test with multiple inputs.
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n // 2 + 1):
```

```
        if n % i == 0:
            return False
    return True
# Testing the function with multiple inputs
print(is_prime(7))   # True
print(is_prime(10))  # False
print(is_prime(2))   # True
print(is_prime(15))  # False
print(is_prime(1))   # False
print(is_prime(0))   # False
print(is_prime(-5))  # False
```

Output:



Explanation:

The Java method is converted into a Python function by removing static typing and using Python's simple syntax with indentation. The logic remains the same, using a loop to check divisibility and returning True or False accordingly.

Task 3: Pseudocode to Python Implementation

Prompt:

Convert the above pseudocode into equivalent Python code using bubble sort.Ensure correct logic and test with sample inputs.

Code:

```
# Pseudocode
"""
START
Input list of numbers
FOR i from 0 to n-1
    FOR j from 0 to n-i-2
        IF list[j] > list[j+1]
            SWAP list[j] and list[j+1]
END FOR
Output sorted list
END
"""
```

```python
#Convert the above pseudocode into equivalent Python code using bubble sort.Ensure correct logic and test with sample inputs.
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the list
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Test the bubble_sort function with sample inputs
if __name__ == "__main__":
    sample_input = [64, 34, 25, 12, 22, 11, 90]
    sorted_list = bubble_sort(sample_input)
    print("Sorted list is:", sorted_list)
# Output should be: Sorted list is: [11, 12, 22, 25, 34, 64, 90]
```

Output:

```
  False
PS C:\Users\lilac\Downloads\SQLITE EXPLORER> & C:\Users\lilac\AppData\Local\Programs
ac/Downloads/SQLITE EXPLORER/java to python.py"
Sorted list is: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\lilac\Downloads\SQLITE EXPLORER>
```

Explanation:

The pseudocode is converted into Python by implementing nested loops and swapping elements using Python syntax. The bubble sort algorithm repeatedly compares adjacent elements and sorts the list step by step.

Task 4: Optimize Queries

Prompt:

Convert the above SQL query into equivalent Pandas code

Code:

```python
# SQL Query (for reference)
"""
SELECT name, salary
FROM employees
WHERE salary > 50000;
"""

# Convert the above SQL query into equivalent Pandas code
import pandas as pd
# Assuming we have a DataFrame 'employees' with columns 'name' and 'salary'
# Example DataFrame creation (for demonstration purposes)
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
}
employees = pd.DataFrame(data)
```

```
# Equivalent Pandas code
result = employees[employees['salary'] > 50000][['name', 'salary']]
print(result)
```

Output:

```
  ac/Downloads/SQLITE EXPLORER/task4.py"
      name  salary
  0   Alice   60000
  2 Charlie   70000
PS C:\Users\lilac\Downloads\SQLITE EXPLORER>
```

Explanation:

The SQL query is converted into Pandas by filtering rows using conditions and selecting required columns. Boolean indexing is used in Pandas to replicate the WHERE clause and column selection.

Task 5: Real-Time Application: Algorithm Translation Across Languages

Prompt:

Convert the above Python linear search algorithm into equivalent C code. Ensure proper handling of arrays, loops, and function structure.

Code:

Python Code:

```
# Python Code

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Test
arr = [10, 20, 30, 40, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

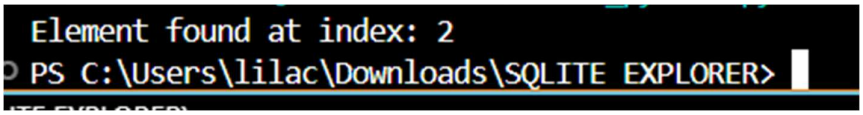
Converted C Code:

```
#Convert the above Python linear search algorithm into equivalent C code. Ensure proper
handling of arrays, loops, and function structure.
#include <stdio.h>
int linear_search(int arr[], int size, int key) {
```

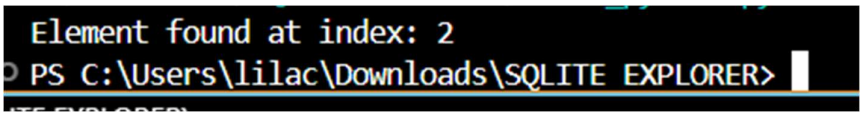
```c
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            return i; // Return the index of the found element
        }
    }
    return -1; // Return -1 if the element is not found
}
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int key = 30;
    int size = sizeof(arr) / sizeof(arr[0]);

    int result = linear_search(arr, size, key);

    if (result != -1) {
        printf("Element found at index: %d\n", result);
    } else {
        printf("Element not found\n");
    }

    return 0;
}
```

Output:

Python:



C:



Explanation:

The Python linear search algorithm is translated into C by using arrays, loops, and functions with explicit data types. Unlike Python, C requires manual handling of array size and uses printf for output.